

GHOSTSTUDIO AI

Merger Blueprint

AI Media Intelligence OS + GhostStudio AI + Reference Repos

Comprehensive Brainstorm, Logic Audit, and Implementation Plan

Version 1.0 | May 2026

Table of Contents

1. Executive Summary

Overview of the merger strategy and key decisions

2. Repository Analysis Summary

Deep-dive findings from all 5 repos

3. Critical Issues & Logic Audit

Bugs, security gaps, and broken logic found

4. Merger Architecture

How the two main repos will be merged

5. Multi-Agent System Design

Full agent orchestration with 24+ specialized agents

6. Puppeteer/ChromeDriver Integration

Real-time browser interaction layer

7. Reference Feature Integration

Features from youtube-heatmap-clipper, Pixelle-Video, etc.

8. Database Schema Merger

Unified 25+ table schema from both repos

9. Security Hardening Plan

Authentication, authorization, input validation

10. Implementation Roadmap

Phase-by-phase execution plan

1. Executive Summary

This document presents a comprehensive merger blueprint for combining GhostStudio AI (the target repository) with AI Media Intelligence OS (the source repository), while integrating valuable features from three additional reference repositories: youtube-heatmap-clipper, Clipper-AI, and Pixelle-Video. The fourth reference repo (Auto-Tiktok-Affiliate-AI) could not be accessed as it appears to be a private repository. The merger will be executed as an upgrade to the GhostStudio AI repository, preserving its existing structure while dramatically enhancing its capabilities.

The core philosophy of this merger is to create a single, extraordinary project that embodies the "True Loop" concept from AI Media Intelligence OS: Signal, Interpretation, Content, Distribution, Audience, Analytics, Memory, and Strategy Update. This loop represents the complete lifecycle of intelligent content creation and distribution, where every piece of data feeds back into the system to improve future output. GhostStudio AI already has the scaffolding for this vision with its 16 agents, OS dashboard, and pipeline architecture, but most of the critical functionality is currently stubbed or returns placeholder data.

The merger addresses three fundamental problems discovered during the logic audit. First, security is critically weak: most API routes have no authentication, password hashing uses SHA-256 instead of bcrypt, and mass assignment vulnerabilities exist across multiple endpoints. Second, core functionality is non-functional: content generation, video rendering, browser automation, and scheduler processing all return stub data without actually executing. Third, the architecture has structural issues: the OS dashboard is a 3000-line single-file component, duplicate store exports create confusion, and the codebase lacks any test coverage whatsoever.

1.1 Key Merger Decisions

Decision	Choice	Rationale
Target Repository	ghoststudio-ai	More complete scaffold with 16 agents, Puppeteer stubs, video/heatmap engines
Database	Prisma (merged schema)	Combine 21 models from GS + 13 from AMI; add TikTok, Heatmap, Video tables
Agent System	24+ specialized agents	Expand from 16 stubs to 24+ fully functional agents with real LLM calls

Browser Automation	Puppeteer (primary)	Already in GS codebase; add ChromeDriver fallback for complex scenarios
Video Engine	Pixelle-Video pipeline	Much more mature than GS stub; has ComfyUI, Playwright rendering, TTS
Heatmap Engine	youtube-heatmap-cli	Working Python microservice; integrate into FastAPI heatmap-engine
Dashboard	Modular component split	Break 3000-line OS page into per-tab components
Auth	bcrypt + session auth on all routes	Fix critical security gap; add middleware auth checks

2. Repository Analysis Summary

Five repositories were cloned and deeply analyzed. Each repository was examined for its directory structure, dependencies, tech stack, API routes, database schema, code quality, and integration potential. The following table provides a high-level comparison of the key metrics across all repositories.

Metric	ghoststudio-ai	ai-media-intel-os	yt-heatmap	Pixelle-Video	Clipper-AI
Language	TypeScript/Python	TypeScript	Python	Python	None (empty)
DB Models	21	13	0	0	0
Agents	16 (stubs)	10 (stubs)	0	0	0
API Routes	25+	18+	7	18	0
Auth	Weak	Broken	None	None	N/A
Tests	0	0	0	0	0
Functional	Scaffold	Scaffold	Working	Working	Empty

2.1 GhostStudio AI (Target Repo)

GhostStudio AI is the most complete scaffold of the two main repositories. It has 21 Prisma models covering users, workspaces, content items, video projects, heatmap jobs, scheduler jobs, memory entries, energy tracking, analytics, and subscriptions. The project includes 16 AI agents (draft, humanic, scoring, SEO, publish, memory, strategy, clip, heatmap, script, voice, image, repurpose, tagging, video-compose, and browser agents), though most return placeholder data. The OS dashboard features 8 tabs (Content Pipeline, Video Studio, Viral Lab, Scheduler, Memory, Analytics, Energy, and Browser Lab) but is implemented as a monolithic 3000-line single-file component.

The codebase also includes two Python FastAPI microservices: a heatmap-engine for YouTube heatmap scanning and clipping, and a video-engine for video generation and rendering. Both are functional scaffolds but lack complete implementations. The project uses Next.js 16 with App Router, Zustand for state management, TanStack Query for server state, NextAuth v4 for authentication,

3. Critical Issues & Logic Audit

The logic audit uncovered 23 critical and significant issues across the two main repositories. These issues must be resolved as part of the merger, as they represent fundamental problems that would prevent the system from functioning correctly in production. The issues are categorized by severity and repository below.

3.1 Security Critical (Must Fix First)

#	Issue	Repo	Impact	Fix
S1	No auth on most API routes	Both	Anyone can access/modify any workspace data	Add getSession checks to all routes
S2	SHA-256 password hashing	GS	Brute-force attacks trivial	Replace with bcrypt/argon2
S3	Mass assignment vulnerability	GS	Attackers can modify workspaceId, scores, status	Whitelist allowed fields in PUT handlers
S4	No input validation on most routes	Both	Malicious data passed directly to Prisma	Add Zod schemas to all endpoints
S5	Auth references non-existent DB fields	AMI	OAuth sign-in crashes at runtime	Add missing Subscription model and plan field
S6	Path traversal in file server	Pixelle	Read arbitrary files from server	Validate paths with resolve + startswith check

3.2 Logic Critical (Core Functionality Broken)

#	Issue	Repo	Impact	Fix
L1	Content generation does not actually generate	GS	Content pipeline is non-functional	Wire generate route to AI orchestrator pipeline
L2	Video generation pipeline does not execute	GS	Video studio is non-functional	Integrate Pixelle-Video pipeline as backend

L 3	Browser routes are no-ops	GS	Browser Lab tab does nothing	Implement real Puppeteer session management
L 4	Scheduler process does not execute jobs	Both	Jobs queue but never run	Wire processJob() to actual agent execution
L 5	Content DNA never injected into AI prompts	AMI	Generated content ignores workspace personality	Read settingsJson and inject into system prompts
L 6	Duplicate scoring implementations	AMI	Two conflicting scoring paths	Consolidate to single 4-dimension scorer
L 7	Energy audience_exhaustion falls through to wrong function	AMI	Two fatigue concepts conflated	Add dedicated trackAudienceExhaustion()

3.3 Architecture Issues

#	Issue	Repo	Fix
A 1	3000-line single-file OS dashboard	GS	Split into per-tab component files
A 2	Duplicate useAppStore exports	AMI	Consolidate into single store
A 3	Two competing dashboard systems	AMI	Remove /dashboard/ routes; use single /os page
A 4	OS dashboard uses mock data, not TanStack hooks	GS	Replace mock data with real API hooks
A 5	Pricing discrepancy (landing vs Stripe config)	GS	Align prices across all references
A 6	ESLint effectively disabled	GS	Re-enable key rules incrementally
A 7	In-memory rate limiting	GS	Use Redis-backed rate limiter for production
A 8	No test coverage anywhere	Both	Add Jest + Playwright tests for critical paths

4. Merger Architecture

The merger follows a "best of both" strategy: GhostStudio AI serves as the structural foundation (more complete scaffold, more agents, more features), while AI Media Intelligence OS contributes its conceptual architecture (True Loop, Content DNA, 7-layer design), documentation, and working implementations that GhostStudio lacks. Reference repos contribute specific working components: youtube-heatmap-clipper for the heatmap engine, Pixelle-Video for the video engine, and Clipper-AI provides only conceptual guidance.

4.1 What We Keep from GhostStudio AI

- Project structure, package.json, Next.js 16 App Router setup
- 21 Prisma models (expanded to 25+ with merged additions)
- 16 agent stubs (expanded to 24+ with full implementations)
- OS dashboard concept with 8 tabs (split into modular components)
- Auth system with NextAuth v4 (fixed and hardened)
- Stripe integration (with pricing fix)
- Landing page with Framer Motion animations
- Puppeteer browser automation structure (fully implemented)
- TanStack Query hooks (connected to real data)
- Python microservices architecture (heatmap + video engines)

4.2 What We Take from AI Media Intelligence OS

- True Loop philosophy: Signal, Interpretation, Content, Distribution, Audience, Analytics, Memory, Strategy Update
- Content DNA system with workspace-scoped voice, tone, audience, perspective, values, avoid patterns, and signature moves
- 7-layer architecture: Presentation, Orchestration, Agents, Engine, Memory, Scheduler, Persistence
- AI Orchestrator with model tier routing (cheap/mid/premium) per task type
- Memory system with reinforcement learning, pattern detection, decay, and platform behavior aggregation
- Content scoring with 4 dimensions (Quality 30%, Humanic 30%, SEO 20%, Trust 20%)

- Energy system with 5 fatigue categories and time-based natural decay
- WordPress publisher with exponential backoff retry
- Publisher Factory pattern with 9 platform registry
- Daily autonomous cycle for job processing, content scheduling, and cleanup
- 15+ documentation files covering all system aspects

4.3 What We Take from Reference Repos

- youtube-heatmap-clipper: Heatmap extraction algorithm (ambil_most_replayed), FFmpeg clip generation pipeline with multi-ratio crop, faster-whisper subtitle generation, split-crop for gaming facecam, progress estimation with easing
- Pixelle-Video: Linear video pipeline (8-step Template Method Pattern), ComfyKit workflow abstraction (self-host + RunningHub), HTML frame rendering via Playwright with 29+ templates, TTS service (Edge-TTS + cloud), ffmpeg-python video composition, asset-based pipeline for user-provided media
- Clipper-AI: Conceptual reference only (AI text clipping, smart summarization) - must be built from scratch

4.4 Merged Architecture Diagram

The merged system follows a 7-layer architecture with the True Loop at its core:

Layer	Name	Components
7	Presentation	Next.js App Router + shadcn/ui + Modular Tab Components + Landing Page
6	Orchestration	AI Orchestrator (model tier routing) + True Loop Engine + Pipeline Runner
5	Agents	24+ Specialized Agents (Draft, Humanic, SEO, Repurpose, Scoring, Memory, Strategy, Clip, Heatmap, Script, Voice, Image, Video, Browser, TikTok, etc.)
4	Engine	Content DNA + Scoring Engine + Energy System + Video Pipeline (Pixelle) + Heatmap Pipeline (YT)
3	Memory	Reinforcement Learning Memory + Pattern Detection + Decay + Platform Behavior + Context Injection

2	Scheduler	Persistent Job Queue + Priority + Locking + Retry + Dead Letter + Daily Cycle + BullMQ (Redis)
1	Persistence	Prisma ORM (25+ models) + PostgreSQL + Redis Cache + Cloudflare R2 Storage

5. Multi-Agent System Design

The merged system will feature 24+ specialized AI agents, each with a clearly defined responsibility, model tier assignment, and integration point. Every agent will make real LLM calls via the z-ai-web-dev-sdk, with the AI Orchestrator routing tasks to the appropriate agent based on job type and model tier. The model tier system uses three levels: "cheap" for high-volume, low-complexity tasks (gpt-4o-mini), "mid" for balanced tasks (claude-3.5-sonnet), and "premium" for complex reasoning tasks (claude-3-opus).

5.1 Complete Agent Registry

#	Agent	Tier	Responsibility	Source
1	Draft Agent	premium	Generate canonical content from ideas with DNA context	Both
2	Humanic Agent	premium	Rewrite content to pass AI detection using 10 Humanic Rules	Both
3	SEO Agent	mid	Generate SEO pack: meta, keywords, schema, headings	Both
4	Repurpose Agent	mid	Adapt content for each platform with tone adjustment	Both
5	Scoring Agent	mid	4-dimension scoring: Quality, Humanic, SEO, Trust	Both
6	Memory Agent	cheap	Store/retrieve/reinforce memory entries with pattern detection	Both
7	Strategy Agent	premium	Analyze analytics + memory to update content strategy	Both
8	Tagging Agent	cheap	Auto-classify content with topic/format/niche/tone tags	GS

9	Script Agent	mid	Generate video scripts with scene breakdowns	GS
10	Voice Agent	cheap	TTS voice selection and narration generation	GS
11	Image Agent	mid	AI image generation via ComfyUI/Replicate	GS
12	Video Compose Agent	mid	Compose video scenes with templates + media + TTS	GS
13	Publish Agent	cheap	Execute publishing to platforms via Publisher Factory	GS
14	Clip Agent	mid	Extract viral clips from heatmap data	GS
15	Heatmap Agent	cheap	Scan YouTube heatmaps and extract engagement segments	GS+YT
16	Browser Agent	cheap	Execute Puppeteer actions for live testing and publishing	GS
17	TikTok Agent	mid	TikTok content creation, scheduling, affiliate linking	New
18	Thumbnail Agent	mid	Generate click-optimized thumbnails for videos	New
19	Caption Agent	cheap	Generate captions/subtitles via Whisper + translation	YT+Pixelle
20	Trend Agent	mid	Detect trending topics and signals from web/social data	New
21	Review Agent	premium	Human-in-the-loop review: approve/reject/modify content	New
22	Format Agent	cheap	Format content to platform-specific requirements	AMI

2 3	Summary Agent	ch ea p	Generate summaries and key point extraction	AMI
2 4	QA Agent	mi d	Pre-publish quality assurance checks and validation	New

5.2 Agent Orchestration Flow

The AI Orchestrator receives tasks from the Scheduler or direct API calls, resolves the appropriate agent and model tier, injects Content DNA and Memory context into the system prompt, executes the LLM call via z-ai-web-dev-sdk, parses the structured response, logs the result to SystemLog, and updates Memory entries via reinforcement learning. Each agent follows a consistent pattern: receive task context, call LLM with DNA-injected prompt, parse response, update database, return result. The orchestrator supports sequential pipelines (draft, then humanic, then SEO, then score) and parallel execution for independent agents (e.g., scoring 4 dimensions simultaneously).

Content Pipeline Flow

Idea (Signal) - Draft Agent generates canonical content using DNA context and memory-informed hooks and topics. The draft agent pulls the top-performing hooks and topics from the memory system to guide its generation, ensuring that new content builds on proven patterns rather than starting from scratch.

Humanic Rewrite - Humanic Agent applies the 10 Humanic Rules to make the content feel genuinely human-written. This includes breaking repetitive sentence structures, removing generic transitions, adding personality and conviction, using contractions, creating unexpected transitions, and eliminating hedging language. The agent also checks alignment with the workspace Content DNA values and avoid patterns.

SEO Optimization - SEO Agent generates a comprehensive SEO pack including meta title and description, focus and secondary keywords, heading structure recommendations, internal link suggestions, schema markup, and readability score. The SEO agent considers the platform target and adjusts its recommendations accordingly.

Scoring - Scoring Agent evaluates the content on 4 dimensions (Quality 30%, Humanic 30%, SEO 20%, Trust 20%) and produces a composite score. Based on the score, the content is either auto-scheduled for publication (score 80+), flagged for human review (60-79), or rejected for rewrite (below 60). Trust scoring includes

6. Puppeteer/ChromeDriver Integration

The browser automation layer is one of the most critical additions to the merged system. It enables real-time screen interaction, live testing of published content, automated posting to platforms that lack APIs, visual regression testing, and screenshot capture for analytics. The current implementation in GhostStudio AI has three API routes (`/api/browser`, `/api/browser/interact`, `/api/browser/screenshot`) that are all placeholders returning stub data. The `browser-manager.ts`, `page-interactions.ts`, `platform-actions.ts`, `testing-runner.ts`, and `live-preview.ts` modules exist but are not connected to the API routes.

6.1 Architecture

Component	Technology	Purpose
Browser Manager	Puppeteer (primary)	Session lifecycle, pool management, health checks
Page Interactions	Puppeteer + CDP	Click, type, scroll, wait, evaluate JS on pages
Platform Actions	Puppeteer + custom scripts	Auto-login, auto-post, auto-schedule on social platforms
Testing Runner	Puppeteer + Jest	E2E testing, visual regression, accessibility checks
Live Preview	Puppeteer + WebSocket	Real-time browser view streamed to dashboard
ChromeDriver Fallback	Selenium WebDriver	Fallback for complex scenarios Puppeteer cannot handle

6.2 Browser Session Management

The Browser Manager will implement a pool-based session management system. Each browser instance runs in headless mode (configurable via `PUPPETEER_HEADLESS` env var) with a maximum concurrent instance limit (default: 5, configurable via `PUPPETEER_MAX_INSTANCES`). Sessions are created per-workspace with automatic cleanup after idle timeout (default: 5 minutes, configurable via `PUPPETEER_IDLE_TIMEOUT`). The manager tracks session state (active, idle, error) and provides health check endpoints for monitoring.

Each browser session maintains its own context with cookies, `localStorage`, and session storage persisted across navigation. This enables the system to log into

platforms once and reuse the session for subsequent actions. The platform actions module provides pre-built scripts for common operations: login to WordPress admin, create draft, upload media, publish post; login to TikTok Creator, upload video, set caption and tags; login to YouTube Studio, upload video, set metadata. These scripts use the page interactions module for low-level operations and include error recovery with retry logic.

The Live Preview feature streams the browser viewport to the OS dashboard via WebSocket, allowing users to watch the automation in real-time and intervene manually if needed. The browser screenshot API captures high-resolution screenshots at any point during automation for analytics, thumbnails, or visual regression testing. The Testing Runner integrates with Jest to run E2E test suites against the live application, including visual comparison tests that detect UI regressions between deployments.

6.3 API Endpoints (Fully Implemented)

Endpoint	Method	Function
/api/browser/session	POST	Create new browser session with platform context
/api/browser/session/[id]	GET/DELETE	Get session status / Terminate session
/api/browser/navigate	POST	Navigate session to URL with wait conditions
/api/browser/interact	POST	Execute click/type/scroll/select actions on page
/api/browser/screenshot	POST	Capture screenshot (full page or element)
/api/browser/execute	POST	Run custom JavaScript in page context
/api/browser/platform-action	POST	Execute pre-built platform automation script
/api/browser/test	POST	Run E2E test suite and return results
/api/browser/live	WebSocket	Stream live browser viewport to dashboard

7. Reference Feature Integration

The reference repositories contribute specific working components that significantly enhance the merged system. Each integration is designed to be modular, so that the core system can function without the optional components while benefiting from them when available.

7.1 YouTube Heatmap Clipper Integration

The youtube-heatmap-clipper provides the most valuable integration: a working YouTube engagement data extraction pipeline. The core algorithm (ambil_most_replayed) scrapes the YouTube watch page HTML and extracts heatMarkerRenderer data using regex, filters by intensity score (configurable threshold, default 0.40), sorts by engagement, and returns structured segment data. This replaces the current heatmap-engine stub with a fully functional implementation.

The integration plan involves refactoring the Flask-based clipper into a FastAPI module that fits within the existing heatmap-engine microservice. The global mutable state will be replaced with a ClipJob class that holds all configuration per-job, making it thread-safe. The in-memory job store will be replaced with the Prisma-backed SchedulerJob system for persistence and reliability. The Whisper transcription language will be made configurable (not hardcoded to Indonesian). Ffmpeg command construction will be abstracted into a structured builder to prevent injection and improve testability.

7.2 Pixelle-Video Integration

Pixelle-Video provides a mature video generation pipeline that dramatically surpasses the current video-engine stub. The integration will replace the video-engine FastAPI microservice with a Pixelle-Video-based implementation that inherits its pipeline architecture, ComfyKit workflow abstraction, TTS service, HTML frame rendering, and ffmpeg-python video composition. The key components to integrate include:

Component	Current GS Status	Pixelle Contribution
Pipeline Architecture	Stub only	8-step LinearVideoPipeline with Template Method Pattern

Image Generation	Replicate (stub)	ComfyKit: FLUX, SDXL, SD3.5 via self-host or RunningHub
Video Generation	Stub only	WAN 2.1/2.2, LTX2 workflows via ComfyKit
TTS	Edge-TTS (stub)	Edge-TTS + Index-TTS + Spark TTS via ComfyUI
Frame Rendering	None	Playwright HTML rendering with 29+ templates
Video Composition	FFmpeg CLI (stub)	ffmpeg-python with intelligent duration matching
Asset Pipeline	None	AI analyzes uploaded media and generates matching scripts

7.3 TikTok Affiliate Integration

The Auto-Tiktok-Affiliate-AI repository could not be cloned (likely private), but based on its name and the user requirements, the merged system should include TikTok automation capabilities. These will be built from scratch as a new TikTok Agent and Platform Publisher. The TikTok Agent will handle content adaptation for TikTok format (9:16 vertical, under 60 seconds, hook-first structure), affiliate link generation and tracking, hashtag research and optimization, and scheduling for optimal posting times. The TikTok Publisher will extend the Publisher Factory with browser-based automation via Puppeteer, since TikTok lacks a public content posting API. This publisher will use the browser automation layer from Section 6 to log into TikTok Creator, upload videos, set captions and tags, and manage posting schedules.

8. Database Schema Merger

The merged database schema combines the 21 models from GhostStudio AI with the 13 models from AI Media Intelligence OS, resolving conflicts and adding new tables for TikTok, enhanced video pipeline, and improved analytics. The result is a unified 25+ model schema that preserves all functionality from both repos while adding new capabilities.

8.1 Schema Unification Decisions

Model	Source	Changes
User	Both (merged)	Add plan field from AMI; add Subscription model; fix auth references
Workspace	Both (merged)	Keep settingsJson with Content DNA; add automationConfig field
ContentItem	Both (merged)	Keep all GS fields; add sourceNotes, sourceType from AMI
ContentVariant	Both (merged)	Add TikTok to platform enum; add metadataJson
SeoData	Both (same)	Keep as-is
ContentTag	Both (same)	Keep as-is
PublishJob	Both (merged)	Add isDryRun, responsePayload from AMI
SchedulerJob	Both (merged)	Expand jobType enum; add TikTok job types
AnalyticsEvent	Both (merged)	Add source field (auto/manual/api); add rawPayload
MemoryEntry	Both (merged)	Add isActive flag; add contextJson; add source types
EnergyEntry	Both (merged)	Add audience_exhaustion and hook_repetition categories
VideoProject	GS (kept)	Keep all GS fields; add pipelineType, assetPaths

VideoScene	GS (kept)	Add templateName from Pixelle; add htmlContent
VideoAsset	GS (kept)	Keep as-is
VideoTemplate	GS+Pixelle	Add htmlTemplate field; add category; add isPremium
VideoRenderJob	GS+Pixelle	Add engineUrl, outputUrl; add progress tracking
HeatmapClipJob	GS+YT	Add transcriptData, clipReason, peakScore from YT
Subscription	New	Add from AMI design (was missing from schema); Stripe integration
ApiCredential	AMI (kept)	Keep for platform API tokens and credentials
SystemLog	Both (same)	Keep as-is; essential for audit trail
TikTokCampaign	New	TikTok affiliate campaigns, links, tracking, revenue
BrowserSession	New	Active Puppeteer sessions with state tracking
ContentTest	New	A/B test results, visual regression, E2E outcomes

9. Security Hardening Plan

Security is the highest priority for the merger. The current codebases have critical security gaps that must be addressed before any feature development. The following plan outlines the security measures that will be implemented as part of the merger, organized by priority level.

9.1 Priority 1: Authentication & Authorization

- Replace SHA-256 with bcrypt: The current password hashing in `/api/auth/signup` uses SHA-256 with a salt, which is not designed for password hashing. bcrypt automatically handles salting and has a configurable work factor that makes brute-force attacks computationally expensive. The migration will update existing hashes on next login.
- Add session checks to all API routes: Every API route (except `/api/auth/signup` and `/api/health`) must verify the user session using `getServerSession`. Routes that modify data must also verify workspace membership. A middleware helper function will be created to DRY this pattern across all routes.
- Fix AMI auth crash: The AMI `auth.ts` references `dbUser.plan` and creates a Subscription that does not exist in the Prisma schema. The merged schema will include the Subscription model and plan field on User, resolving this runtime crash.
- Implement role-based access control: The User model already has a role field (operator/admin/viewer). This will be enforced at the API level: operators can create/edit content, admins can manage users and settings, viewers can only read. The middleware will check role permissions for each route.

9.2 Priority 2: Input Validation

- Add Zod schemas to all API routes: Currently only `/api/auth/signup` validates input. Every POST and PUT endpoint will receive a Zod schema that validates request body structure, types, and constraints before data reaches Prisma. This prevents malformed data, injection attacks, and mass assignment vulnerabilities.
- Whitelist updatable fields: Instead of passing `req.body` directly to Prisma update operations, each route will explicitly destructure only the allowed fields. This prevents mass assignment attacks where an attacker could modify `workspaceId`, `scores`, or `status` fields.

10. Implementation Roadmap

The merger will be executed in 5 phases, each building on the previous phase and delivering a functional increment. This phased approach allows for testing and validation at each stage, reducing the risk of integration failures. Each phase includes specific deliverables, estimated duration, and dependencies.

Phase 1: Security & Foundation (Week 1)

This phase addresses the critical security issues and establishes the merged project foundation. All subsequent work depends on a secure, stable base. The deliverables include: replacing SHA-256 with bcrypt for password hashing, adding session authentication checks to all API routes, implementing Zod input validation on all endpoints, fixing the AMI auth crash by adding the Subscription model, whitelisting updatable fields in PUT handlers, merging the Prisma schemas into a unified 25+ model schema, and resolving the duplicate store exports. By the end of this phase, the system should be secure against the identified vulnerabilities and have a unified database schema.

Phase 2: Agent System & AI Pipeline (Week 2-3)

This phase brings the AI system to life by implementing all 24+ agents with real LLM calls. The deliverables include: wiring the AI Orchestrator to actually route tasks to agents, implementing each agent with proper system prompts injected with Content DNA and Memory context, connecting the content generation pipeline (Draft, Humanic, SEO, Score) to the `/api/content/[id]/generate` route, connecting the scheduler process endpoint to the actual agent execution pipeline, implementing the reinforcement learning memory system with pattern detection and context injection, and integrating the energy system with the publish flow to enforce fatigue limits. By the end of this phase, the True Loop should be functional: ideas flow through the content pipeline, get scored, published, and feed back into memory for strategy updates.

Phase 3: Video & Heatmap Engines (Week 3-4)

This phase replaces the stub video and heatmap engines with working implementations based on the reference repos. The deliverables include: integrating Pixelle-Video pipeline as the video-engine FastAPI microservice,

Risk	Probability	Impact	Mitigation
YouTube HTML structure changes break heatmap extraction	Medium	High	Add fallback to YouTube API; monitor extraction success rate
ComfyUI workflow compatibility issues	Medium	Medium	Pin workflow versions; add health check endpoint
Puppeteer detection by social platforms	High	Medium	Use stealth plugin; rotate user agents; add human-like delays
LLM API rate limits during batch processing	Medium	Medium	Implement request queuing with backoff; use cheap tier for bulk tasks
Database migration issues during merger	Low	High	Use Prisma migrate with explicit rollback scripts; test on copy first

End of Merger Blueprint